Production-Level Data Analytics Project Plans

Project 1: Customer Churn Prediction & Lifetime Value (LTV) Engine

Project Information

A predictive analytics system designed for telecommunications or subscription-based businesses. This engine ingests historical customer demographic data, billing information, and service usage metrics to identify customers at high risk of cancellation (churn). Additionally, it calculates the predictive Customer Lifetime Value (LTV) to help marketing teams prioritize high-value retention campaigns.

Data Source

- **Primary Source:** The Telco Customer Churn Dataset (available via Kaggle or IBM Watson analytics repositories).
- **Characteristics:** Contains over 7,000 rows of customer data, including tenure, monthly charges, contract types, internet service details, and churn status.

Tech Stack

- **Languages:** Python, SQL.
- **Data Engineering & Storage:** PostgreSQL (Data Warehouse), SQLAlchemy.
- **Machine Learning & Analysis:** Pandas, Scikit-Learn, XGBoost, SHAP (for model explainability).
- **API Layer:** FastAPI (to serve predictions to other internal applications).
- **Visualization:** Apache Superset or Metabase.

Expected Impact

Directly decreases customer acquisition costs by proactively retaining existing users. Optimizes the marketing budget by segmenting users based on their predicted LTV, ensuring resources are spent on the most profitable demographics rather than uniform, generalized campaigns.

4-Week Development Timeline

- **Week 1: Data Ingestion & Exploratory Data Analysis (EDA)**
    - Day 1-2: Setup PostgreSQL database and load the Telco dataset.
    - Day 3-5: Perform extensive EDA using Pandas and Seaborn to identify correlations between contract types, tenure, and churn.

    - Day 6-7: Handle missing values, encode categorical variables, and establish the baseline analytics report.
- **Week 2: Feature Engineering & Predictive Modeling**
    - Day 1-3: Engineer new features (e.g., average monthly usage vs. base charge).
    - Day 4-6: Train classification models (Logistic Regression, Random Forest, XGBoost) and evaluate using precision, recall, and F1-score.
    - Day 7: Implement SHAP values to explain feature importance for business stakeholders.
- **Week 3: LTV Calculation & API Development**
    - Day 1-3: Develop regression models to forecast the expected lifetime revenue of active customers.
    - Day 4-7: Build a FastAPI service with endpoints for single-customer inference and batch prediction processing.
- **Week 4: Visualization & Deployment**
    - Day 1-3: Connect Apache Superset/Metabase to the database and API.
    - Day 4-5: Build interactive dashboards showing global churn risk and segmenting users by LTV.
    - Day 6-7: Containerize the application using Docker and finalize technical documentation.

Project 2: Real-Time Financial Fraud Detection Pipeline

Project Information

A streaming data analytics pipeline designed to monitor financial transactions in real-time. This system evaluates incoming transactions against historical baselines and user behavioral profiles to instantly flag anomalous, potentially fraudulent activities before funds are settled.

Data Source

- **Primary Source:** PaySim Synthetic Dataset for Mobile Money Transactions (available via Kaggle).
- **Characteristics:** Contains over 6 million simulated financial transactions reflecting normal and fraudulent behaviors, including transfer amounts, origin/destination balances, and transaction types.

Tech Stack

- **Languages:** Python, Scala (optional for Spark).
- **Streaming Engine:** Apache Kafka.
- **Data Processing:** Apache Spark (Structured Streaming / MLlib).
- **Database:** Cassandra or MongoDB (High write-throughput NoSQL for fast logging).
- **Monitoring & Dashboarding:** Grafana, Prometheus.

Expected Impact

Significantly minimizes financial losses and limits institutional exposure to regulatory penalties. Enhances platform security and maintains customer trust by reducing false positives and accelerating the interception of malicious transactions.

4-Week Development Timeline

- **Week 1: Infrastructure Setup & Data Simulation**
  - Day 1-3: Provision Apache Kafka and Cassandra environments (locally or via Docker).
  - Day 4-7: Write a Python producer script to simulate a continuous stream of transactions from the PaySim dataset into Kafka topics.
- **Week 2: Batch Analysis & Model Training**
  - Day 1-3: Extract historical batch data to train an anomaly detection model (Isolation Forest or Autoencoders).
  - Day 4-6: Evaluate model performance focusing heavily on minimizing false negatives (missed fraud).
  - Day 7: Export and serialize the trained model for integration into the streaming pipeline.
- **Week 3: Real-Time Stream Processing**
  - Day 1-4: Configure Apache Spark Structured Streaming to consume data from Kafka topics in micro-batches.
  - Day 5-7: Apply the pre-trained machine learning model within the Spark pipeline to score transactions on the fly and route flagged transactions to a dedicated "fraud alerts" database.
- **Week 4: Analytics Dashboarding & Optimization**
  - Day 1-3: Connect Grafana to the underlying database to visualize transaction velocity, alert volumes, and systemic latency.
  - Day 4-5: Optimize Kafka partitions and Spark memory tuning to ensure sub-second processing latency.
  - Day 6-7: Finalize system architecture documentation and simulated load testing.

Project 3: Retail Demand Forecasting & Inventory Optimization

Project Information

An automated analytics platform for retail and supply chain teams. It utilizes time-series forecasting to analyze historical sales data, promotional events, and seasonality to accurately predict future product demand at the store and department levels, generating optimal inventory restocking schedules.

Data Source

- **Primary Source:** M5 Forecasting Dataset (Walmart historical sales data, available via Kaggle).

- **Characteristics:** Hierarchical sales data involving item level, department, product categories, and store details over several years, including calendar events and pricing.

Tech Stack

- **Languages:** Python, SQL.
- **Cloud Data Warehouse:** Google BigQuery or Snowflake.
- **Data Transformation:** dbt (Data Build Tool).
- **Forecasting Models:** Facebook Prophet, ARIMA, or LightGBM.
- **Visualization:** Streamlit or Tableau.

Expected Impact

Prevents revenue loss from stockouts on high-demand items while simultaneously reducing warehousing costs associated with overstocking. Improves supply chain efficiency by shifting from reactive purchasing to proactive, data-driven procurement.

4-Week Development Timeline

- **Week 1: Data Architecture & ETL**
  - Day 1-3: Setup Google BigQuery/Snowflake. Write extraction scripts to load the raw M5 dataset.
  - Day 4-7: Implement data quality checks and format dates, sales volumes, and pricing information.
- **Week 2: Data Transformation with dbt**
  - Day 1-3: Configure dbt and connect it to the data warehouse.
  - Day 4-6: Create dbt models to aggregate daily sales into weekly/monthly views and build clean data marts for analysis.
  - Day 7: Document the data lineage and transformation logic within dbt.
- **Week 3: Time-Series Forecasting**
  - Day 1-3: Implement Facebook Prophet to model seasonality, holidays, and trend shifts for high-volume products.
  - Day 4-6: Train LightGBM models for complex, multi-variable forecasting at the granular item level.
  - Day 7: Store forecast outputs (predicted demand vs. actuals) back into the data warehouse.
- **Week 4: Interactive Dashboard & Reporting**
  - Day 1-4: Build a Streamlit web application allowing inventory managers to select stores/categories and view 30-day demand forecasts.
  - Day 5-6: Implement "what-if" scenario inputs (e.g., simulating a 10% price drop) to see impact on predicted volume.
  - Day 7: Final code reviews, access control configuration, and project handover.

REQUIRED PROTOCOL FOR ALL ENGINEERING TEAMS

IMPORTANT INSTRUCTION FOR ALL TEAMS: Before commencing development, every team must create their respective GitHub repository. You are required to push commits daily for every week of the project lifecycle. This commit history will serve as the strict metric for your daily productivity tracking. Without consistent daily commits, your evaluation will not be processed by zaalima. Ensure professional commit messages, logical branch management, and adherence to standard collaborative workflows.